

Snapshot Read - Memtables and Timestamps

In this chapter, you will:

- Refactor your memtable/WAL to store multiple versions of a key.
- Implement the new engine write path to assign each key a timestamp.
- Make your compaction process aware of multi-version keys.
- Implement the new engine read path to return the latest version of a key.

During the refactor, you might need to change the signature of some functions from `&self` to `self: &Arc<Self>` as necessary.

To run test cases,

```
cargo x copy-test --week 3 --day 2
cargo x scheck
```

Note: You will also need to pass everything `<= 2.4` after finishing this chapter.

Task 1: MemTable, Write-Ahead Log, and Read Path

In this task, you will need to modify:

```
src/wal.rs
src/mem_table.rs
src/lsm_storage.rs
```

We have already made most of the keys in the engine to be a `KeySlice`, which contains a bytes key and a timestamp. However, some part of our system still did not consider the timestamps. In our first task, you will need to modify your memtable and WAL implementation to take timestamps into account.

You will need to first change the type of the `SkipMap` stored in your memtable.

```
pub struct MemTable {
    // map: Arc<SkipMap<Bytes, Bytes>>,
    map: Arc<SkipMap<KeyBytes, Bytes>>, // Bytes -> KeyBytes
    // ...
}
```

After that, you can continue to fix all compiler errors so as to complete this task.

MemTable::get

We keep the get interface so that the test cases can still probe a specific version of a key in the memtable. This interface should not be used in your read path after finishing this task. Given that we store `KeyBytes`, which is `(Bytes, u64)` in the skiplist, while the user probe the `KeySlice`, which is `(&[u8], u64)`. We have to find a way to convert the latter to a reference of the former, so that we can retrieve the data in the skiplist.

To do this, you may use unsafe code to force cast the `&[u8]` to be static and use `Bytes::from_static` to create a bytes object from a static slice. This is sound because `Bytes` will not try to free the memory of the slice as it is assumed static.

► Spoilers: Convert u8 slice to Bytes

This was not a problem because what we had before is `Bytes` and `&[u8]`, where `Bytes` implements `Borrow<[u8]>`.

MemTable::put

The signature should be changed to `fn put(&self, key: KeySlice, value: &[u8])` and You will need to convert a key slice to a `KeyBytes` in your implementation.

MemTable::scan

The signature should be changed to `fn scan(&self, lower: Bound<KeySlice>, upper: Bound<KeySlice>) -> MemTableIterator`. You will need to convert `KeySlice` to `KeyBytes` and use these as `SkipMap::range` parameters.

MemTable::flush

Instead of using the default timestamp, you should now use the key timestamp when flushing the memtable to the SST.

MemTableIterator

It should now store `(KeyBytes, Bytes)` and the return key type should be `KeySlice`.

Wal::recover and Wal::put

Write-ahead log should now accept a key slice instead of a user key slice. When serializing and deserializing the WAL record, you should put timestamp into the WAL file and do checksum over the timestamp and all other fields you had before.

The WAL format is as follows:

```
| key_len (exclude ts len) (u16) | key | ts (u64) | value_len (u16) | value |
checksum (u32) |
```

LsmStorageInner::get

Previously, we implement `get` as first probe the memtables and then scan the SSTs. Now that we change the memtable to use the new key-ts APIs, we will need to re-implement the `get` interface. The easiest way to do this is to create a merge iterator over everything we have – memtables, immutable memtables, L0 SSTs, and other level SSTs, the same as what you have done in `scan`, except that we do a bloom filter filtering over the SSTs.

LsmStorageInner::scan

You will need to incorporate the new memtable APIs, and you should set the scan range to be `(user_key_begin, TS_RANGE_BEGIN)` and `(user_key_end, TS_RANGE_END)`. Note that when you handle the exclude boundary, you will need to correctly position the iterator to the next key (instead of the current key of the same timestamp).

Task 2: Write Path

In this task, you will need to modify:

```
src/lsm_storage.rs
```

We have an `mvcc` field in `LsmStorageInner` that includes all data structures we need to use for multi-version concurrency control in this week. When you open a directory and initialize the storage engine, you will need to create that structure.

In your `write_batch` implementation, you will need to obtain a commit timestamp for all keys in a write batch. You can get the timestamp by using `self.mvcc().latest_commit_ts() + 1` at the beginning of the logic, and `self.mvcc().update_commit_ts(ts)` at the end of the logic to increment the next commit timestamp. To ensure all write batches have different timestamps and new keys are placed on top of old keys, you will need to hold a write lock `self.mvcc().write_lock.lock()` at the beginning of the function, so that only one thread can write to the storage engine at the same time.

Task 3: MVCC Compaction

In this task, you will need to modify:

```
src/compact.rs
```

What we had done in previous chapters is to only keep the latest version of a key and remove a key when we compact the key to the bottom level if the key is removed. With MVCC, we now have timestamps associated with the keys, and we cannot use the same logic for compaction.

In this chapter, you may simply remove the logic to remove the keys. You may ignore `compact_to_bottom_level` for now, and you should keep ALL versions of a key during the compaction.

Also, you will need to implement the compaction algorithm in a way that the same key with different timestamps are put in the same SST file, *even if* it exceeds the SST size limit. This ensures that if a key is found in an SST in a level, it will not be in other SST files in that level, and therefore simplifying the implementation of many parts of the system.

Task 4: LSM Iterator

In this task, you will need to modify:

```
src/lsm_iterator.rs
```

In the previous chapter, we implemented the LSM iterator to act as viewing the same key with different timestamps as different keys. Now, we will need to refactor the LSM iterator to only return the latest version of a key if multiple versions of the keys are retrieved from the child iterator.

You will need to record `prev_key` in the iterator. If we already returned the latest version of a key to the user, we can skip all old versions and proceed to the next key.

At this point, you should pass all tests in previous chapters except persistence tests (2.5 and 2.6).

Test Your Understanding

- What is the difference of `get` in the MVCC engine and the engine you built in week 2?
- In week 2, you stop at the first memtable/level where a key is found when `get`. Can you do the same in the MVCC version?
- How do you convert `KeySlice` to `&KeyBytes`? Is it a safe/sound operation?
- Why do we need to take a write lock in the write path?

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Bonus Tasks

- **Early Stop for Memtable Gets.** Instead of creating a merge iterator over all memtables and SSTs, we can implement `get` as follows: If we find a version of a key in the memtable, we can stop searching. The same applies to SSTs.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.